TrendPilot: Local Agentic AI Assistant for Viral Content Generation

Student & Submission Details

Student Name: Uzair Abbas
Student ID: BSAI-23037
Department: Department of Artificial Intelligence
Institute: Information Technology University (ITU), Lahore
Internship Program: AIRI Team — Punjab Information Technology Board (PITB)
Assignment: AI Internship Task 2

Project Repository & Artifact Links

- **GitHub Repository:** https://github.com/UzairAbbas-dev/TrendPilot_Local_Agentic_AI_Assistant

Project Overview & Objective

The Practical Problem

Developers, students, and technical creators often struggle to turn complex engineering projects into engaging, readable content for platforms like LinkedIn, X, and Instagram. Translating technical milestones—such as deploying a computer vision model or building a microservice—into clear hooks, structured explanations, and short-form video scripts is time-consuming.

While conventional chatbots can generate generic drafts, they suffer from three major workflow limitations:

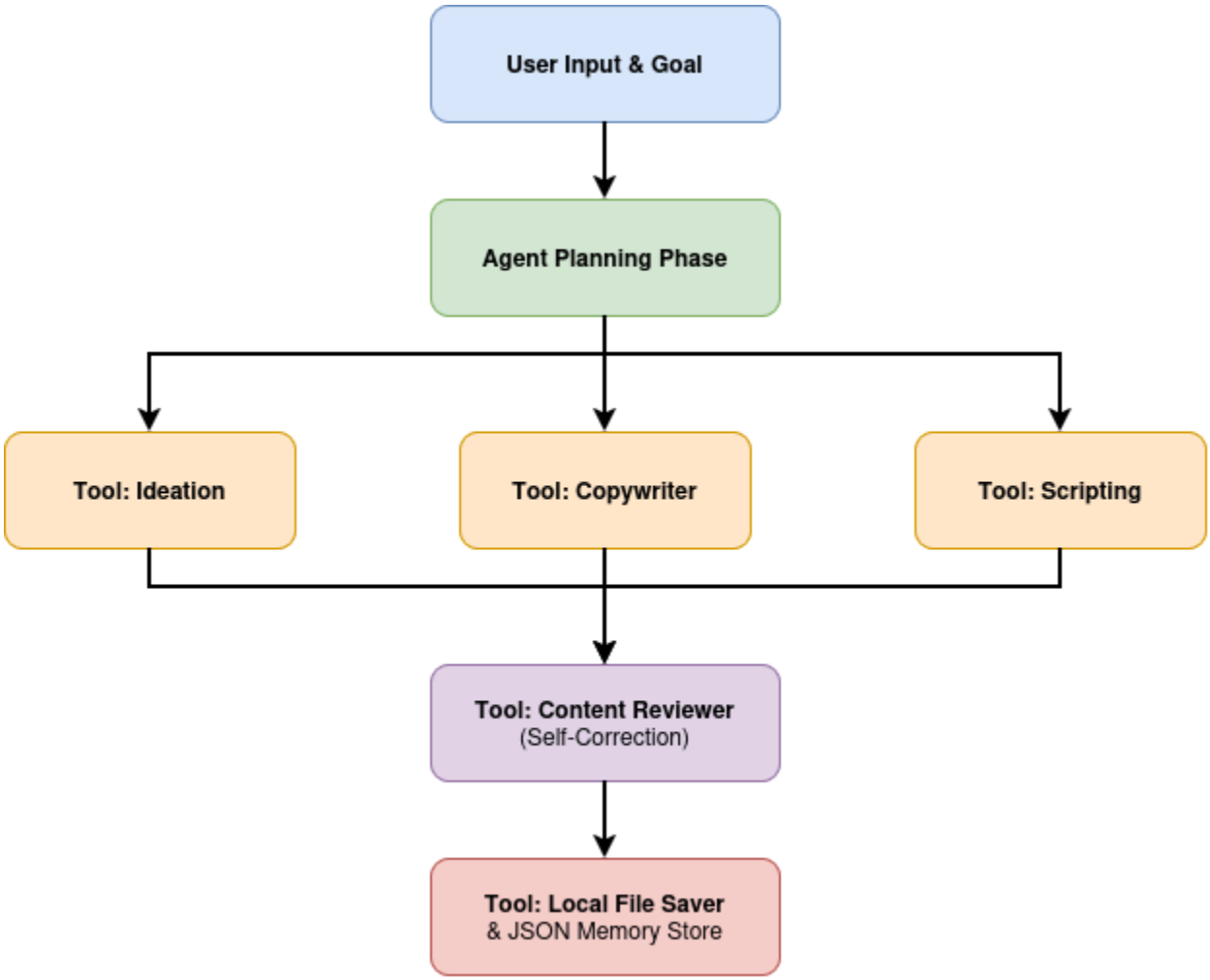
1. **Manual orchestration:** The user has to repeatedly prompt the model for each piece (one prompt for the hook, another for the body, another for hashtags).
2. **Lack of self-correction:** Chatbots output the first draft they produce without evaluating readability or platform appropriateness.
3. **No automated persistence:** The user must manually copy, paste, format, and organize outputs across local folders.

The Agentic Solution: TrendPilot

TrendPilot was built to turn this multi-step manual chore into an autonomous, one-click agentic pipeline. Instead of operating as a passive chat interface, TrendPilot acts as an autonomous content strategist running entirely on local hardware.

When provided with a raw technical topic (e.g., *"YOLOv8 helmet safety detection project"*), target platform, and tone, the agent executes an end-to-end operational lifecycle:

- **Strategic Planning:** It decomposes the request into discrete milestones.
- **Tool Invocation:** It routes specific subtasks to specialized internal tools (Ideation, Copywriting, Video Scripting, Hashtag Generation).
- **Editorial Critique:** A dedicated reviewer tool evaluates the generated hook and body copy against platform standards.
- **Disk & Memory Persistence:** It automatically stores the full markdown artifact to the local filesystem and logs the session in structured JSON memory.



Key Differences: Standard Chatbot vs. AI Agent

To fulfill the core learning objectives of this assignment, the architecture highlights the fundamental boundary between basic conversational models and true agents:

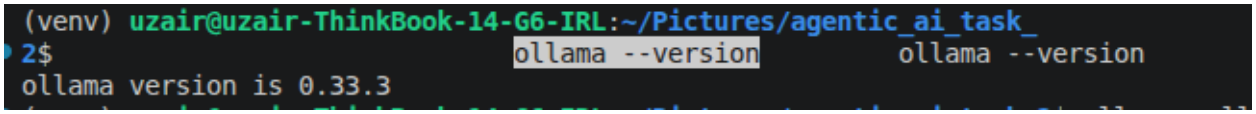
Dimension	Standard Chatbot	TrendPilot Agent
Execution Pattern	Reactive	Autonomous
Task Handling	Generates an entire response in one broad pass.	Decomposes goals into specialized functional modules.
Tool Integration	Limited to plain text completion.	Calls deterministic scripts (regex parser, file I/O, memory updater).
Evaluation Loop	Does not review its own text.	Evaluates its own output and provides targeted QA notes.
State & Storage	Ephemeral; lost once the browser tab closes.	Persists structured markdown files and a JSON run history.

Local Environment Setup & LLM Integration

Hardware and OS Environment

To maintain zero cloud dependencies and ensure total data privacy, the entire system was developed and tested on a personal workstation running Linux:

- **Host Machine:** Lenovo ThinkBook 14 G6 IRL
- **Operating System:** Linux (x86_64 Ubuntu-based environment)
- **Python Runtime:** Python 3.12 within an isolated virtual environment (venv)
- **Local Inference Server:** Ollama daemon serving endpoints over localhost:11434



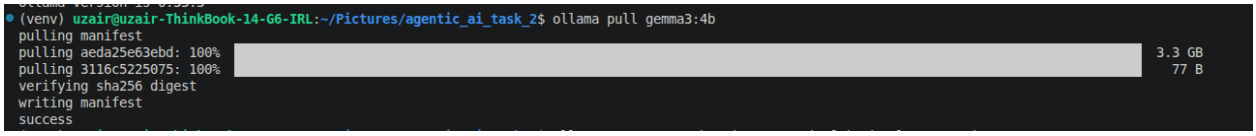
Local LLM Selection & Setup

The assignment recommended small, edge-capable models capable of reasoning on local consumer laptops. For this implementation, **gemma3:4b** was selected as the primary reasoning engine, paired with fallback compatibility for **llama3.2:3b** and **qwen2.5:3b**.
Ollama was installed via the native Linux installation script:
curl -fsSL https://ollama.com/install.sh | sh

After installation, the system service was verified, and the target model was pulled and tested:

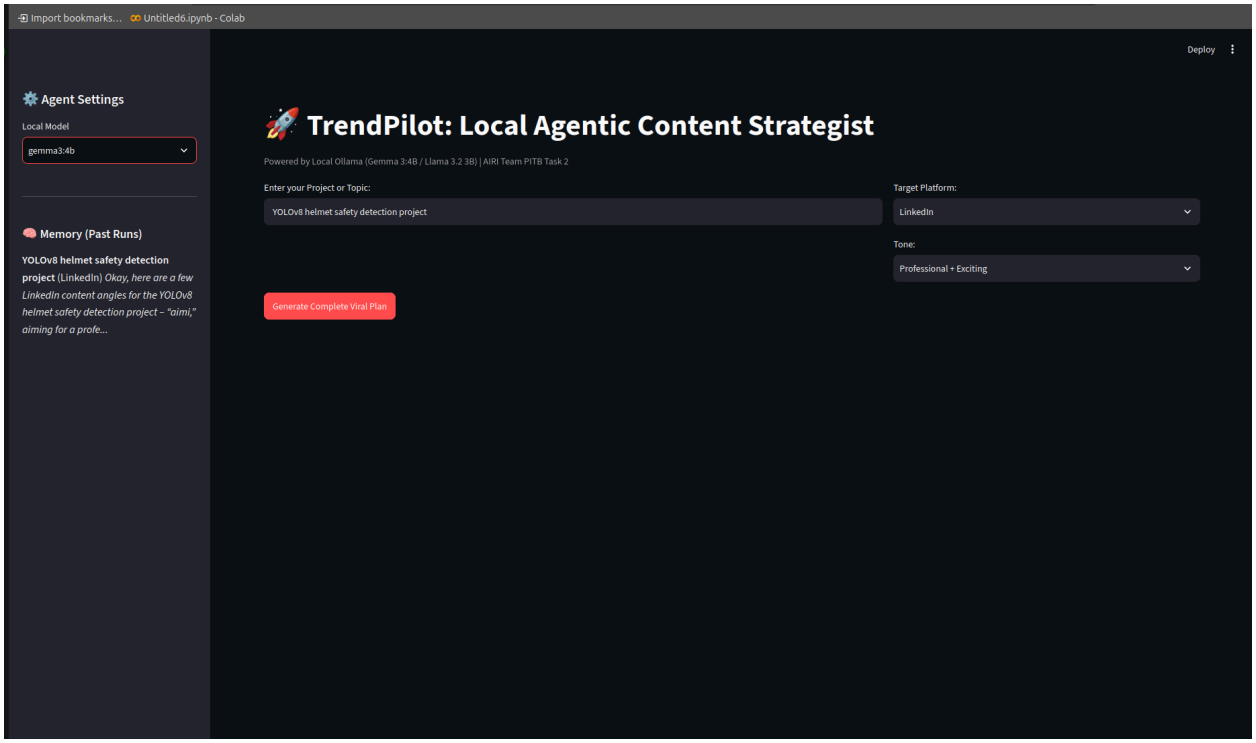
Verify Ollama server status
ollama --version

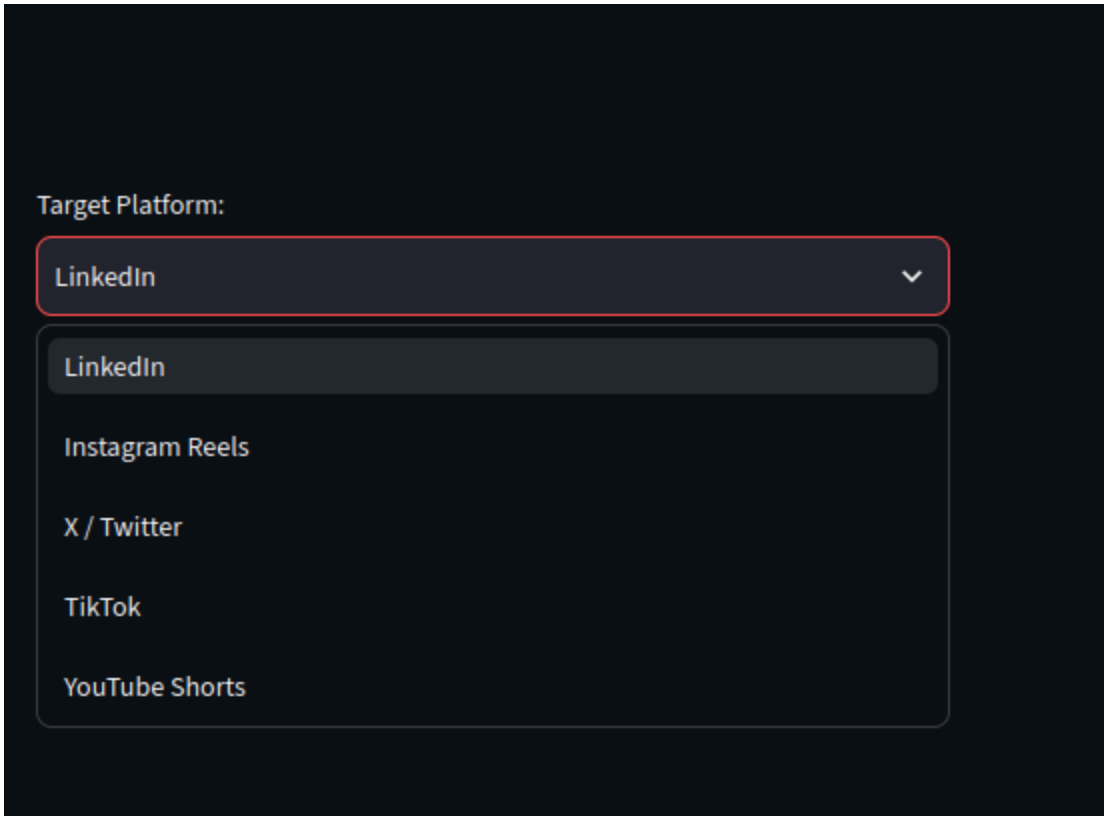
Pull the Gemma 3 4B parameter model
ollama pull gemma3:4b

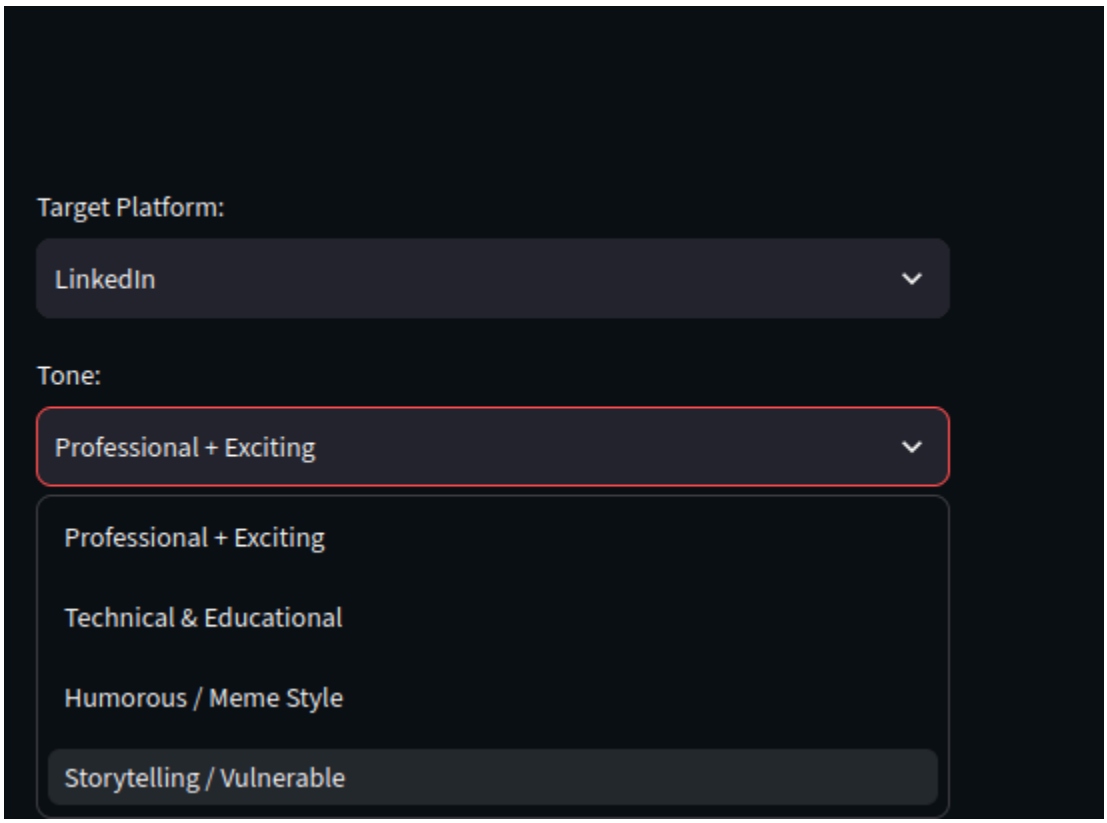


Run baseline verification prompt
ollama run gemma3:4b "Give me 3 viral hooks for AI engineers."

```
(venv) uzair@uzair-ThinkBook-14-G6-IRL:~/Pictures/agentic_ai_task_2$ ollama run gemma3:4b "Give me 3 viral hooks for AI engineers."
Okay, here are 3 viral hooks designed to grab the attention of AI engineers, leaning into current trends and frustrations, with explanations of why they might work:

**1. (Short, Frustrating, Relatable - TikTok/Twitter Focused)**

* **Hook:** “My AI model is *almost* perfect… just needs one more dataset. 😩 #AIProblems #MachineLearning #DataHungry”
* **Why it Works:** This taps into the universal feeling of AI engineers chasing marginal improvements. The emoji adds a relatable, slightly exasperated tone.  Hashtags
target the right communities. It's short enough for quick consumption on platforms like TikTok and Twitter.
* **Potential Viral Element:** People will immediately recognize the struggle and offer their own "almost perfect" stories in the comments.


**2. (Intriguing Question, Technical - LinkedIn/Reddit Focused)**

* **Hook:** “Building a generative AI that *actually* understands nuance?  I’m experimenting with [mention a specific technique – e.g., Retrieval-Augmented Generation,
Contrastive Learning] to move beyond superficial mimicry.  What's your biggest challenge with contextual understanding in LLMs?  Let's discuss. 🧠 #AI #LLMs #GenerativeAI
#DeepLearning”
* **Why it Works:** It poses a genuinely complex, current problem.  Mentioning a specific technique shows you're knowledgeable. Asking a direct question invites engagement
and starts a conversation. LinkedIn and Reddit are prime environments for this kind of detailed discussion.
* **Potential Viral Element:**  The "nuance" angle is a hot topic, and people will want to share their research or experiences with related techniques.



**3. (Bold Statement, Provocative - Instagram/YouTube Shorts Focused)**

* **Hook:** “Stop training models on biased data! 🛑 It's not a bug, it’s a feature... unless you’re actively trying to fix it.  Transparency and ethical AI are
*non-negotiable*. #AIethics #BiasInAI #ResponsibleAI #MachineLearning”
* **Why it Works:** This is a provocative statement that forces a reaction. It directly addresses a major concern in the AI field. Using a stop sign graphic adds visual
impact.  The phrase “It's not a bug, it’s a feature…” adds a layer of cynical humor.
* **Potential Viral Element:**  This will spark debate and discussion about the ethical implications of AI, potentially leading to shares and comments expressing strong
opinions.

---

**Important Notes for AI Engineers Sharing These Hooks:**

* **Authenticity:**  Whatever hook you use, make sure it reflects your actual interests and expertise.
* **Engagement:**  Respond to comments and questions – that's key to building a following.
* **Visuals:**  Pair these hooks with relevant images or short video clips to maximize their impact.

To help me tailor these even further, could you tell me:

*   **Which platforms are you primarily targeting?** (e.g., LinkedIn, Twitter, TikTok, YouTube, Reddit)
*   **What specific area of AI engineering are you most involved in?** (e.g., LLMs, Computer Vision, Reinforcement Learning, Data Engineering for AI)
```




Target Platform:
LinkedIn
LinkedIn
Instagram Reels
X / Twitter
TikTok
YouTube Shorts


Target Platform:
LinkedIn
Tone:
Professional + Exciting
Professional + Exciting
Technical & Educational
Humorous / Meme Style
Storytelling / Vulnerable

Python REST API Integration

Rather than relying on heavy third-party abstractions that obscure the underlying mechanics, communication with Ollama is handled through clean, explicit HTTP POST requests directly to Ollama's native REST endpoint (/api/generate).

```
import requests

OLLAMA_URL = "http://localhost:11434/api/generate"

def call_ollama(prompt: str, model: str = "gemma3:4b") -> str:
    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": 0.7
        }
    }
    # Handled with a robust timeout for local CPU/iGPU execution
    response = requests.post(OLLAMA_URL, json=payload, timeout=300)
    response.raise_for_status()
    return response.json().get("response", "").strip()
```

Critical Real-World Debugging: Request Timeout Handling

During initial development runs on Linux, multi-step content generation frequently crashed with an unhandled exception:
HTTPConnectionPool(host='localhost', port=11434): Read timed out. (read timeout=60)

Root Cause:

When generating dense, structured text (such as a 4-part video storyboard or an exhaustive caption draft), a 4-billion parameter model running locally on CPU/integrated graphics requires more than 60 seconds to complete full token evaluation. The default Python requests timeout of 60 seconds terminated the connection prematurely, leaving empty output files.

Solution Applied:

The socket read timeout in app/tools.py was raised from 60 seconds to 300 seconds (timeout=300). This gave the local inference engine sufficient time to process prompt evaluations, generate tokens smoothly, and return complete payloads without dropping the connection.

Agent Workflow Design

Orchestration Logic

Unlike a simple chat script that passes user input directly to a model and prints the result, TrendPilot utilizes an orchestrated, sequential workflow. The core execution loop is managed by the TrendPilotAgent class in app/agent.py.

The agent treats the Large Language Model (LLM) not just as a text generator, but as a reasoning engine to route data through a predefined pipeline. The workflow operates in four distinct phases:

1. **Phase 1: Strategic Planning:** The agent receives the user's raw inputs (topic, platform, tone) and queries the LLM with a highly constrained PLANNER_PROMPT. The model outlines a 4-step strategic execution plan before any content is actually written. This forces the LLM to establish context and logic upfront.

2. **Phase 2: Multi-Tool Dispatch (Execution):** The agent sequentially invokes specialized tools. Importantly, it passes context between them. For example, the Idea Generator creates viral angles, and the agent takes those specific angles and feeds them as input parameters into the Caption Writer tool.

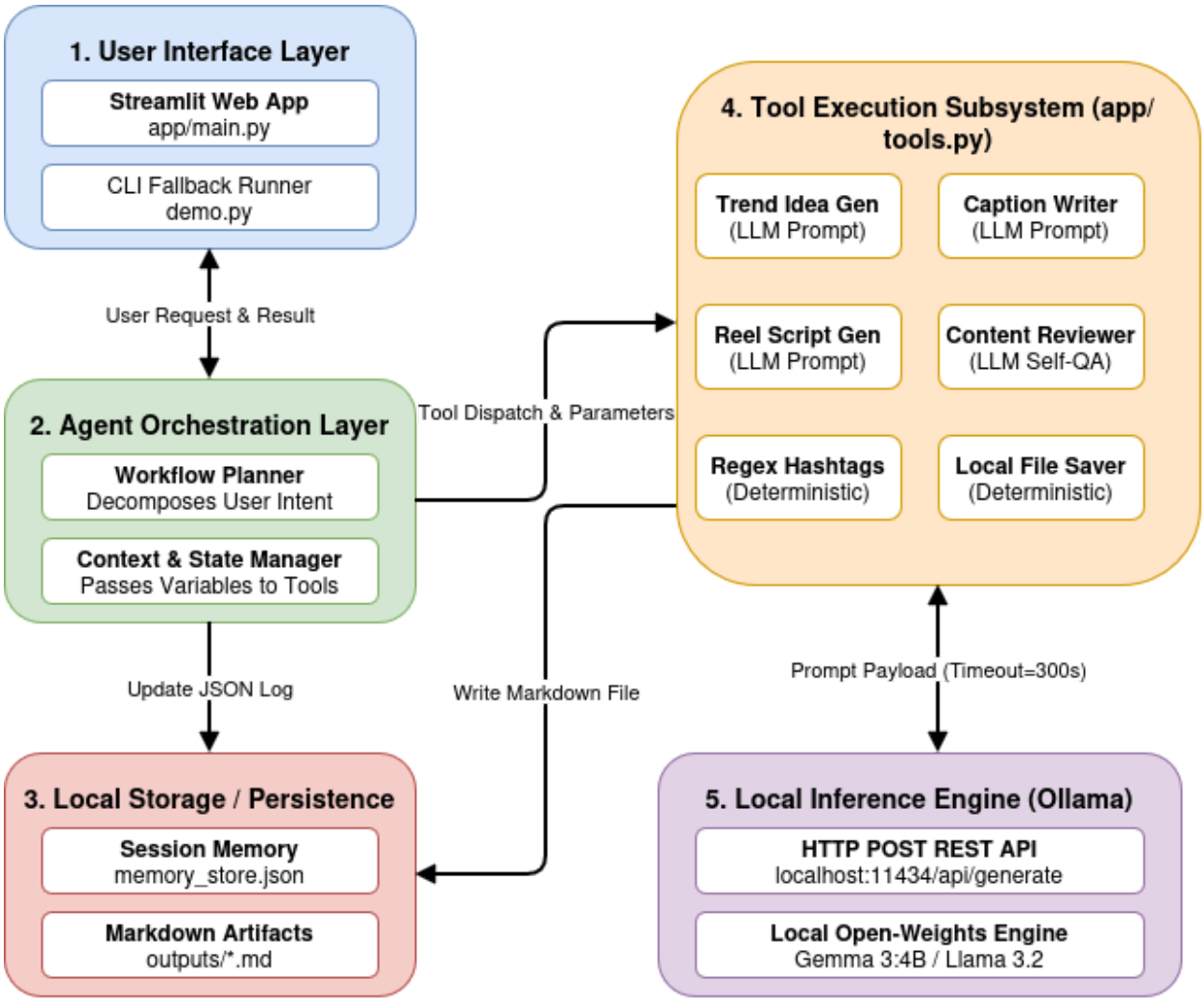
3. **Phase 3: Self-Correction (Review):** The agent takes the newly generated hook and caption and feeds them into the Content Reviewer tool. The agent asks the LLM to evaluate its own prior output against platform best practices and score it out of 10.

4. **Phase 4: Persistence:** The agent gathers all the discrete outputs (Plan, Hook, Caption, Script, Tags, Review) into a final payload dictionary. It triggers the deterministic File Saver tool to write a Markdown file, and pushes a summary to the local JSON memory store.

System Prompting Strategy

Smaller local models (like 3B and 4B parameter models) are highly capable, but they are prone to "chatty" behavior—outputting conversational filler like *"Sure, here are your hashtags!"* instead of just returning the data.

To counteract this, the system uses strict, role-playing system prompts defined in app/prompts.py. Each prompt establishes a persona, enforces output constraints, and dictates the exact layout format (e.g., instructing the script generator to strictly use [0:00-0:05] timestamp brackets).



Implemented Agent Tools

The system relies on six modular tools encapsulated within the TrendPilotTools class. Five tools use LLM inference with targeted prompts, while the sixth is a deterministic Python utility.

1. **Trend Idea Generator:**
    - *Function:* Ingests the topic and target audience to brainstorm 3 distinct, high-engagement content angles.
    - *Mechanism:* Uses a prompt designed to trigger psychological hooks (curiosity, controversy, value).
2. **Caption Writer:**

- *Function:* Drafts the actual post copy.

- *Mechanism:* Takes the angle generated by Tool 1 and outputs a structured post with a distinct scrolling-stopping Hook, a spaced body paragraph, and a clear Call To Action (CTA).

3. **Hashtag Generator (with Regex Sanitization):**

    - *Function:* Generates 8-12 hyper-relevant hashtags grouped from broad to niche.

    - *Mechanism:* Because local models occasionally ignore instructions to omit introductory text, this tool implements a Python regular expression (re.findall(r"#[A-Za-z0-9_]+", raw_output)) to mathematically guarantee only valid hashtags are returned to the final payload.

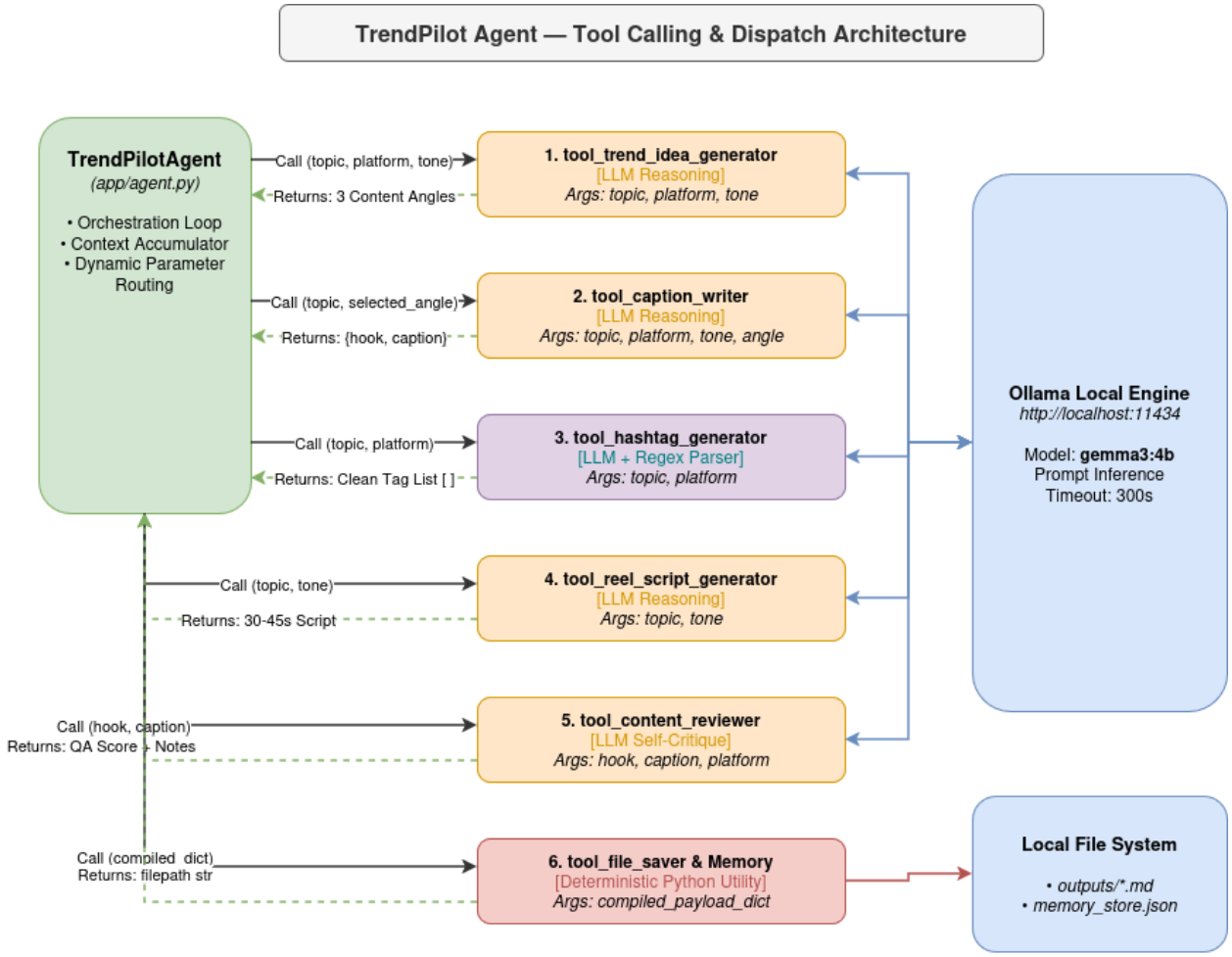
4. **Reel Script Generator:**

    - *Function:* Acts as a short-form video director.

    - *Mechanism:* Generates a 30-45 second video script broken into 4 paced segments (Hook, Problem, Solution, CTA), explicitly noting both Audio and Visual cues for each block.

5. **Content Reviewer (Self-Reflection):**
    - *Function:* Acts as an editorial QA layer.
    - *Mechanism:* Evaluates the drafted caption against platform standards, scores the hook's strength, and provides two actionable bullet points on how a human creator could improve the draft before posting.

6. **Local File Saver (Mandatory Deterministic Tool):**
    - *Function:* Compiles all generated artifacts and saves them permanently to the local disk.
    - *Mechanism:* A pure Python utility (no AI inference). It formats the data into clean Markdown, dynamically creates the outputs/saved_results directory if it doesn't exist, and writes the file.



Memory Management & File Saving

Local State Persistence (Memory)

A defining characteristic of an AI agent is its ability to remember past interactions. Since TrendPilot runs locally, memory cannot be stored in a cloud database.

To solve this, I implemented the AgentMemory class (app/memory.py).

- **Mechanism:** It uses a lightweight JSON file (outputs/memory_store.json) as a local database.
- **Logic:** Every time the agent successfully completes a workflow, it appends a structured dictionary containing the topic, platform, tone, and a truncated summary of the hook to this JSON file. The system is programmed to retain only the last 10 runs to prevent the file from becoming bloated.

- **UI Integration:** The Streamlit sidebar reads this JSON file on load and displays a "Past Runs" history log, allowing the user to see previous campaign strategies across sessions.

Artifact Export (Markdown Saving)

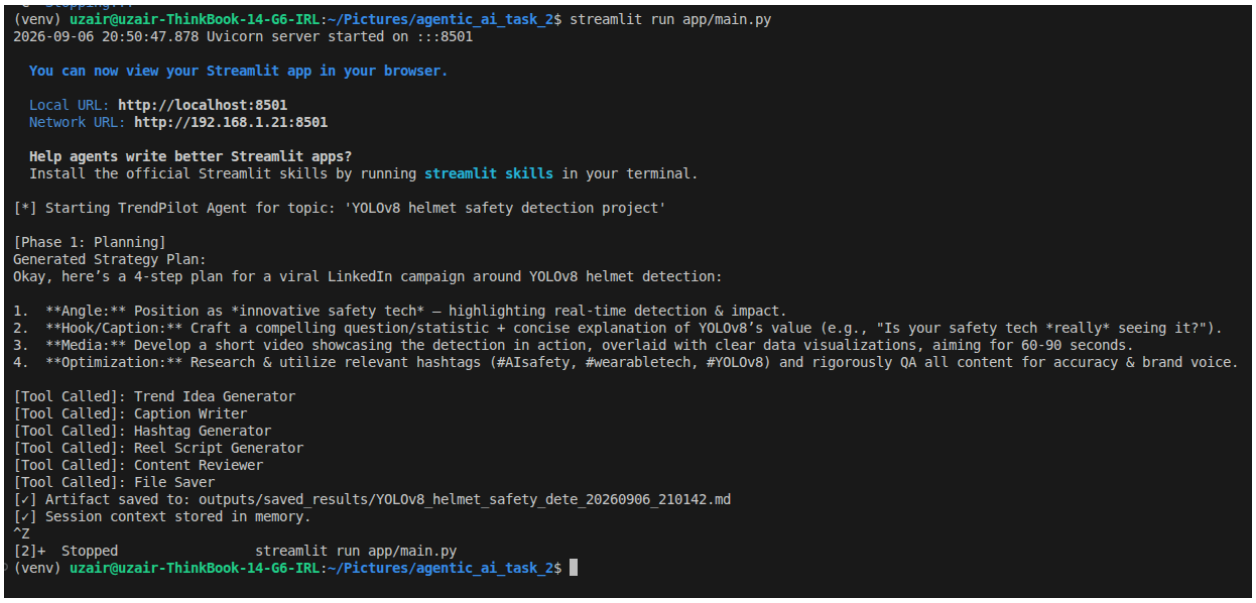
The mandatory file saving requirement is handled by dynamically generating structured .md files.

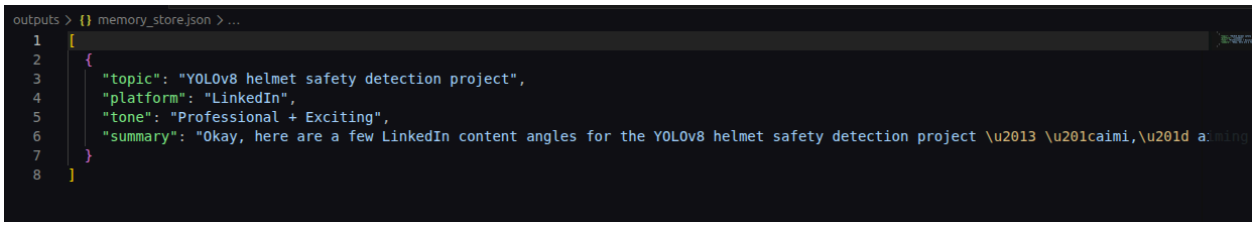
Filename Sanitization Fix:

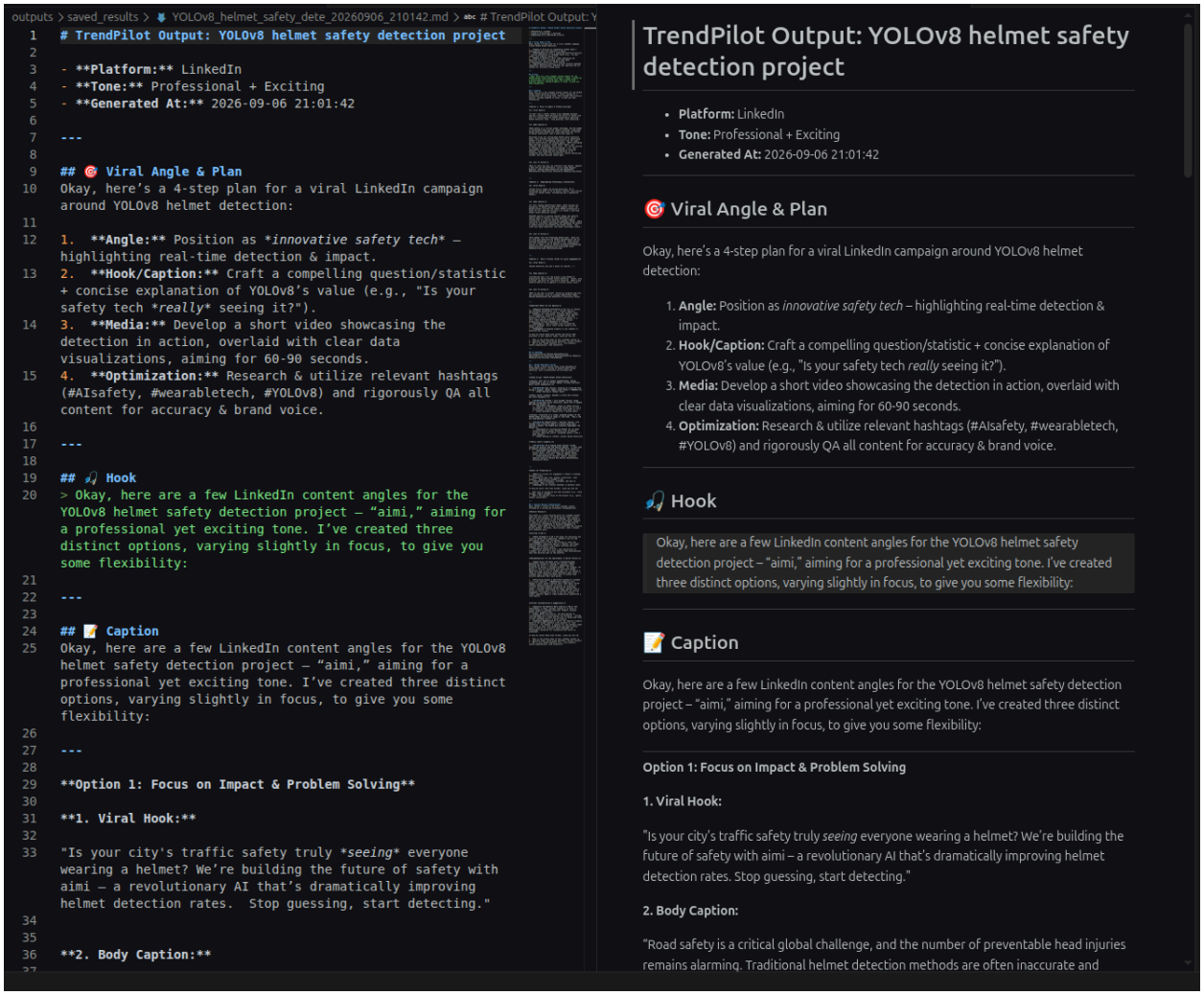
During early testing, passing raw topics (like *"YOLOv8: Real-time detection?"*) directly into the file-saving logic caused OS-level FileNotFoundError crashes because Windows and Linux file systems reject characters like : and ?.
To fix this, the file saver implements strict regex sanitization on the topic string before saving:

```
# Strip special characters and limit length for safe OS file naming
sanitized_title = re.sub(r'[^a-zA-Z0-9_-]', '_', data.get("topic"))[:25]
filename = f"{sanitized_title}_{timestamp}.md"
```

The resulting Markdown files are beautifully formatted with headers, bold text, and blockquotes, ensuring they are immediately readable in VS Code, Obsidian, or GitHub.







User Interface (Streamlit Demo)

While a Command Line Interface (CLI) was implemented via demo.py, a robust web interface was built using **Streamlit** (app/main.py) to provide a more intuitive, visual demonstration of the agent's capabilities.

Interface Architecture

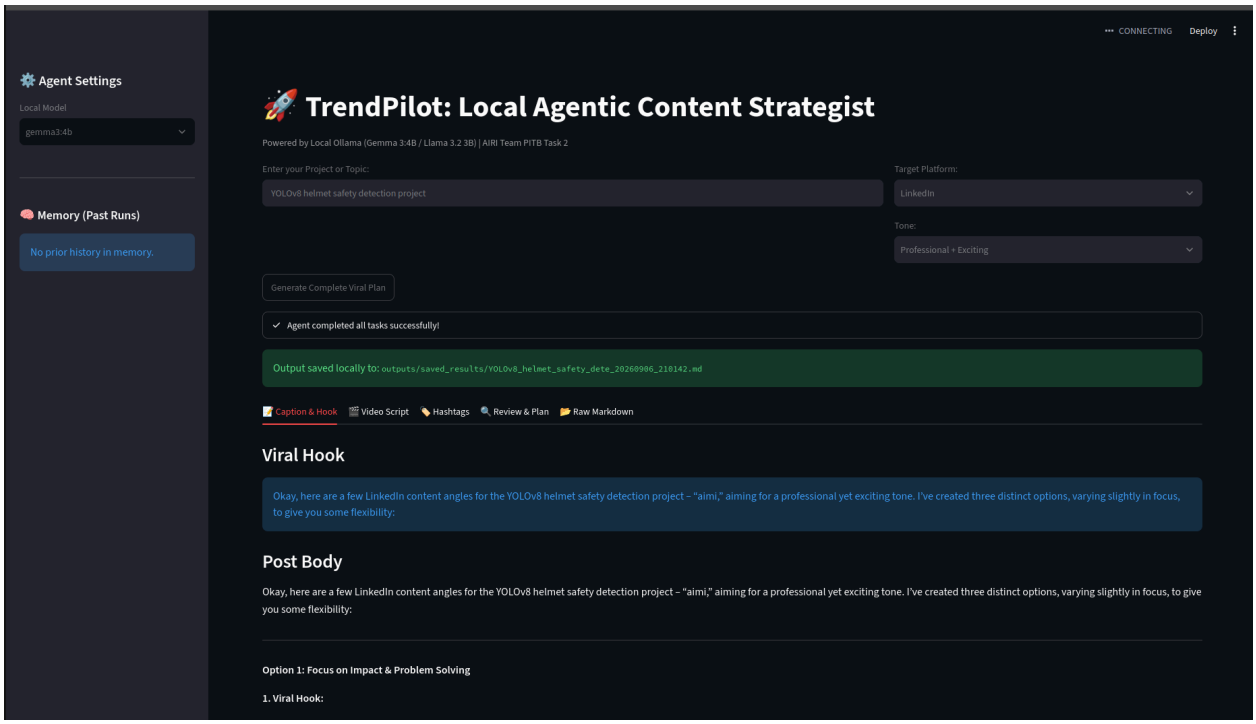
The Streamlit UI is divided into two primary sections:

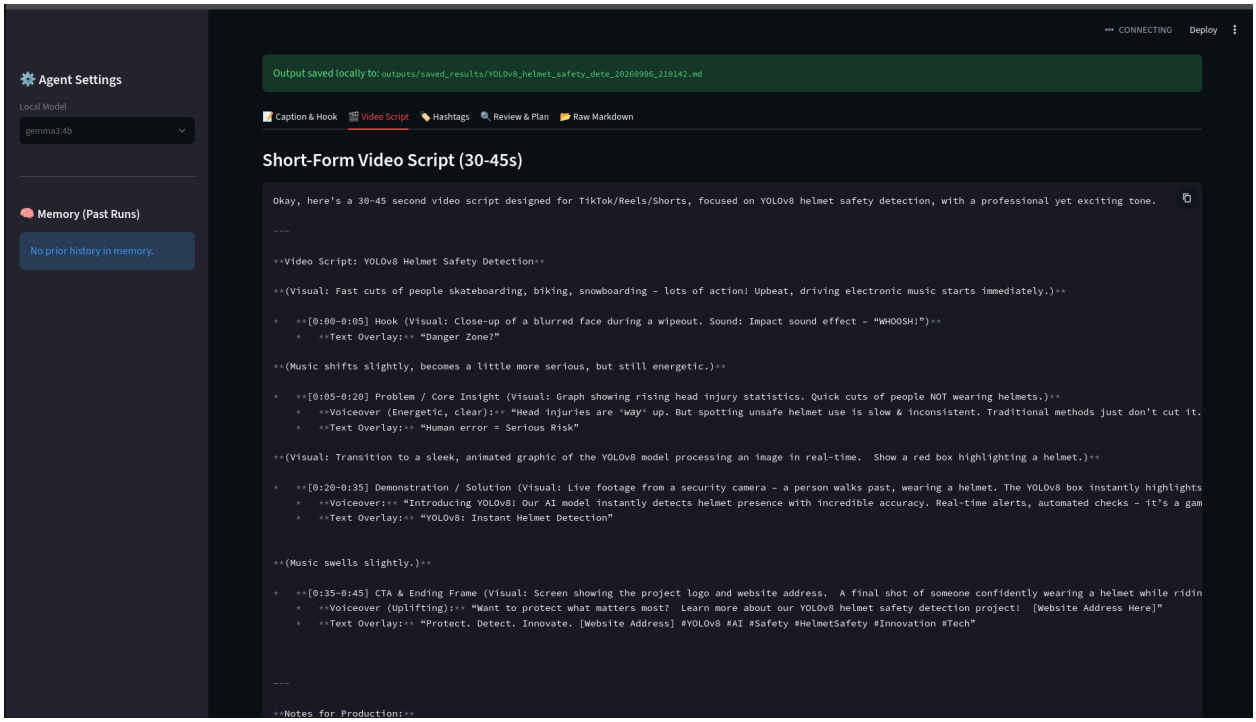
- **Sidebar Controls:** Houses the system-level configurations. Users can dynamically switch between local Ollama models (e.g., gemma3:4b vs llama3.2:3b) via a dropdown. Below the model selector, the UI queries the AgentMemory JSON store to render a visual history of the last 5 generated content campaigns.
- **Main Generation Hub:** Features input fields for the core parameters (Topic, Platform, Tone) and a primary execution button.

Status Tracking & Tabbed Layout

Because local inference can take 1-3 minutes to complete a full 6-tool workflow, a st.status container was implemented. This provides the user with real-time feedback as the agent progresses through Phase 1 (Planning) to Phase 4 (Saving).

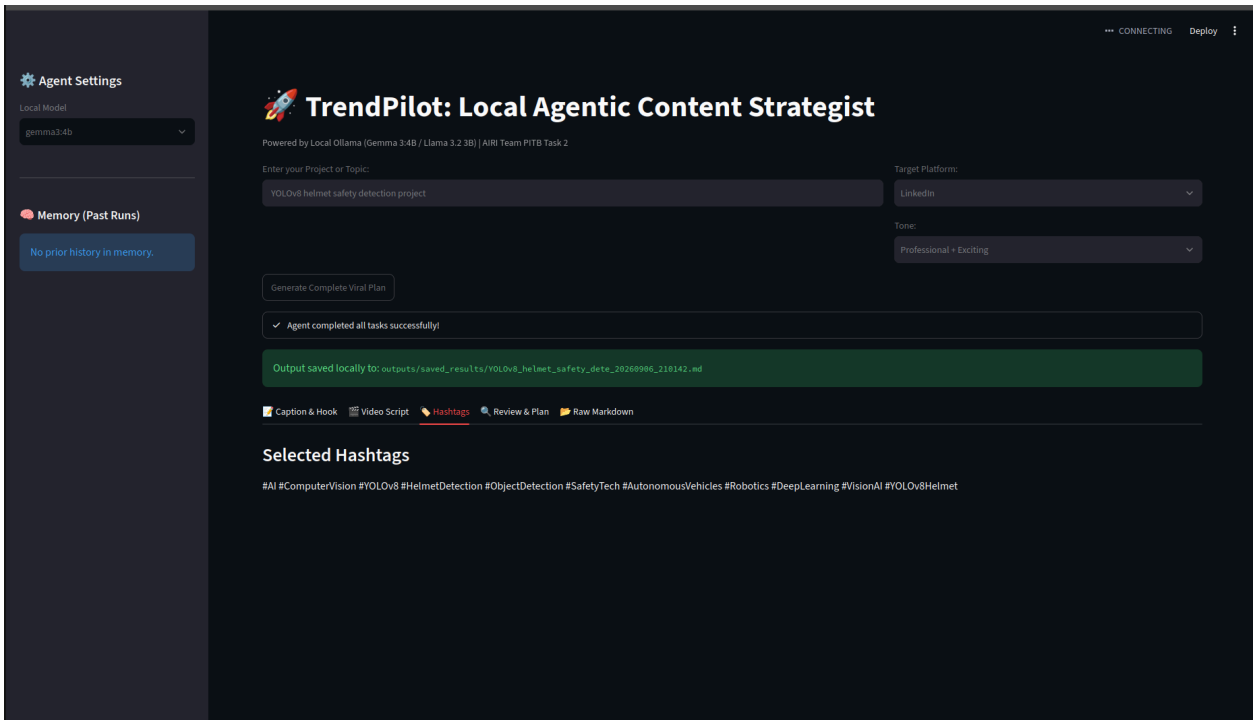
Upon completion, rather than dumping a massive wall of text onto the screen, the interface utilizes Streamlit's st.tabs to cleanly separate the artifacts into digestible views: *Caption & Hook*, *Video Script*, *Hashtags*, *Reviewer QA*, and *Raw Markdown*.

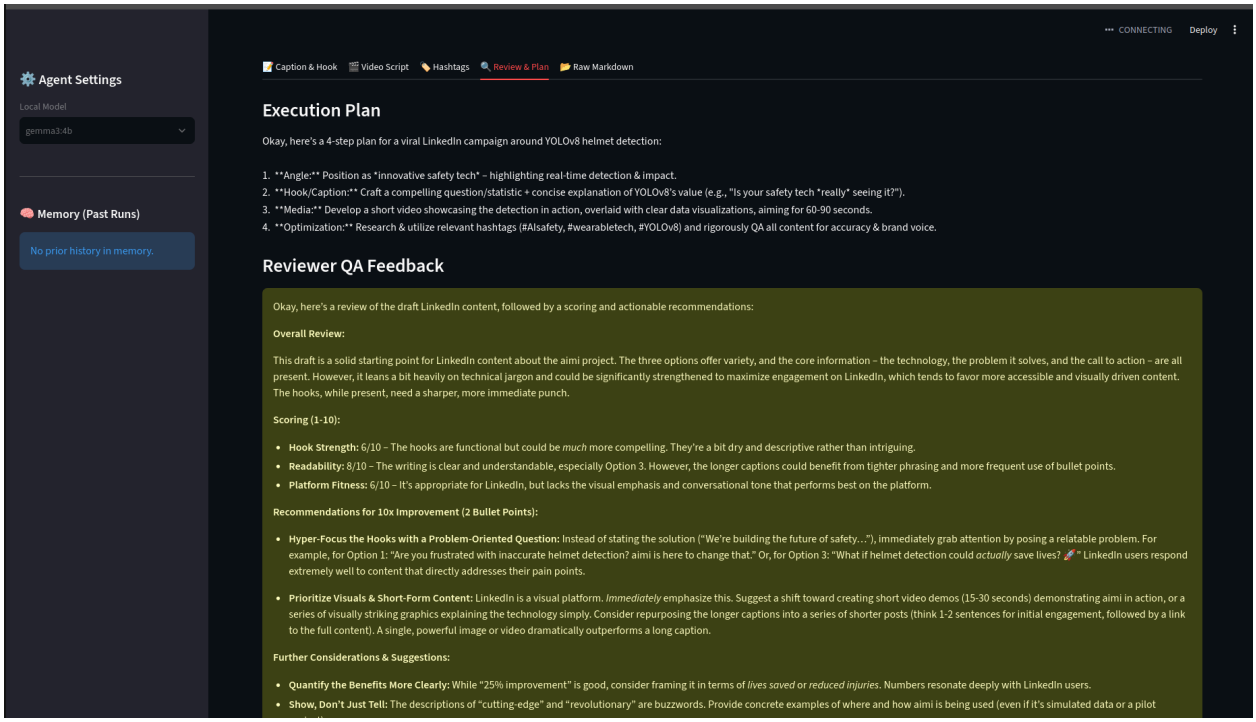


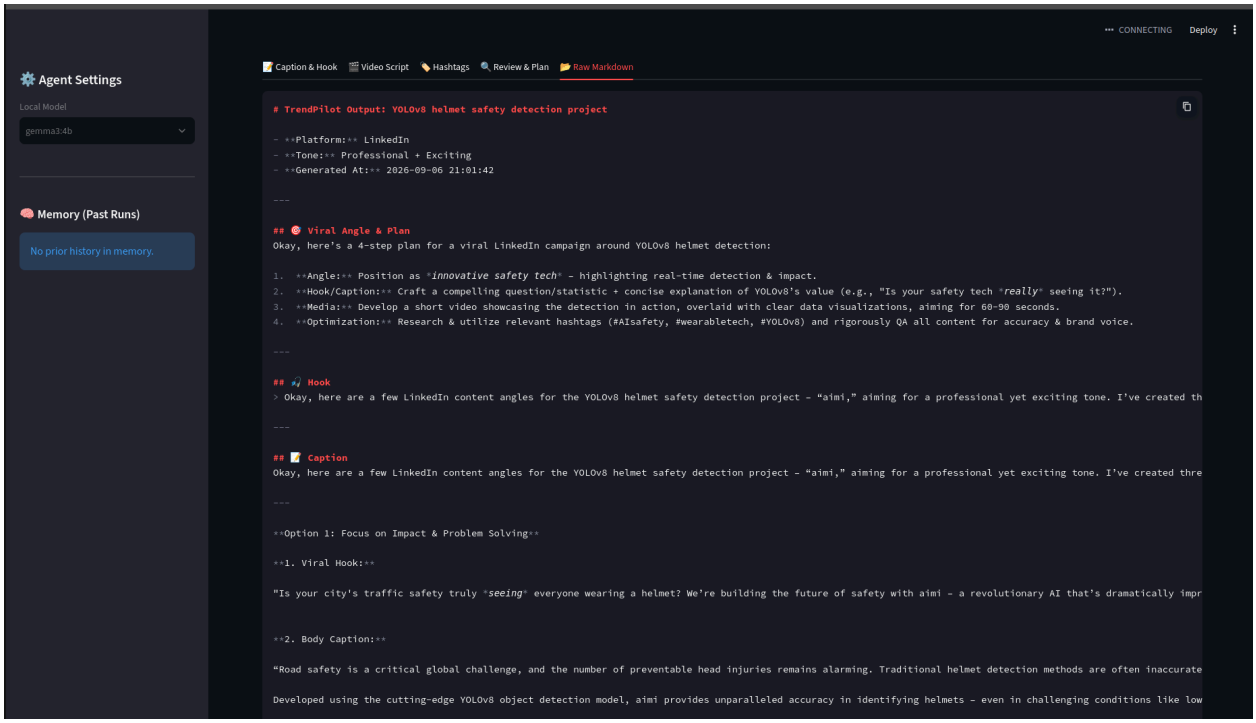

CONNECTING
Deploy
Agent Settings
Local Model
gemma3:4b
Memory (Past Runs)
No prior history in memory.
Output saved locally to: outputs/saved_results/YOLOv8_helmet_safety_dete_20260906_210142.md
Caption & Hook
Video Script
Hashtags
Review & Plan
Raw Markdown
Short-Form Video Script (30-45s)
Okay, here's a 30-45 second video script designed for TikTok/Reels/Shorts, focused on YOLOv8 helmet safety detection, with a professional yet exciting tone.

Video Script: YOLOv8 Helmet Safety Detection
(Visual: Fast cuts of people skateboarding, biking, snowboarding - lots of action! Upbeat, driving electronic music starts immediately.)
* **[0:00-0:05] Hook (Visual: Close-up of a blurred face during a wipeout. Sound: Impact sound effect - "WHOOSH!")**
* **Text Overlay:** "Danger Zone?"
(Music shifts slightly, becomes a little more serious, but still energetic.)
* **[0:05-0:20] Problem / Core Insight (Visual: Graph showing rising head injury statistics. Quick cuts of people NOT wearing helmets.)**
* **Voiceover (Energetic, clear):** "Head injuries are *way* up. But spotting unsafe helmet use is slow & inconsistent. Traditional methods just don't cut it.
* **Text Overlay:** "Human error = Serious Risk"
(Visual: Transition to a sleek, animated graphic of the YOLOv8 model processing an image in real-time. Show a red box highlighting a helmet.)
* **[0:20-0:35] Demonstration / Solution (Visual: Live footage from a security camera - a person walks past, wearing a helmet. The YOLOv8 box instantly highlights
* **Voiceover:** "Introducing YOLOv8! Our AI model instantly detects helmet presence with incredible accuracy. Real-time alerts, automated checks - it's a gam
* **Text Overlay:** "YOLOv8: Instant Helmet Detection"
(Music swells slightly.)
* **[0:35-0:45] CTA & Ending Frame (Visual: Screen showing the project logo and website address. A final shot of someone confidently wearing a helmet while ridin
* **Voiceover (Uplifting):** "Want to protect what matters most? Learn more about our YOLOv8 helmet safety detection project! [Website Address Here]"
* **Text Overlay:** "Protect. Detect. Innovate. [Website Address] #YOLOv8 #AI #Safety #HelmetSafety #Innovation #Tech"

Notes for Production:

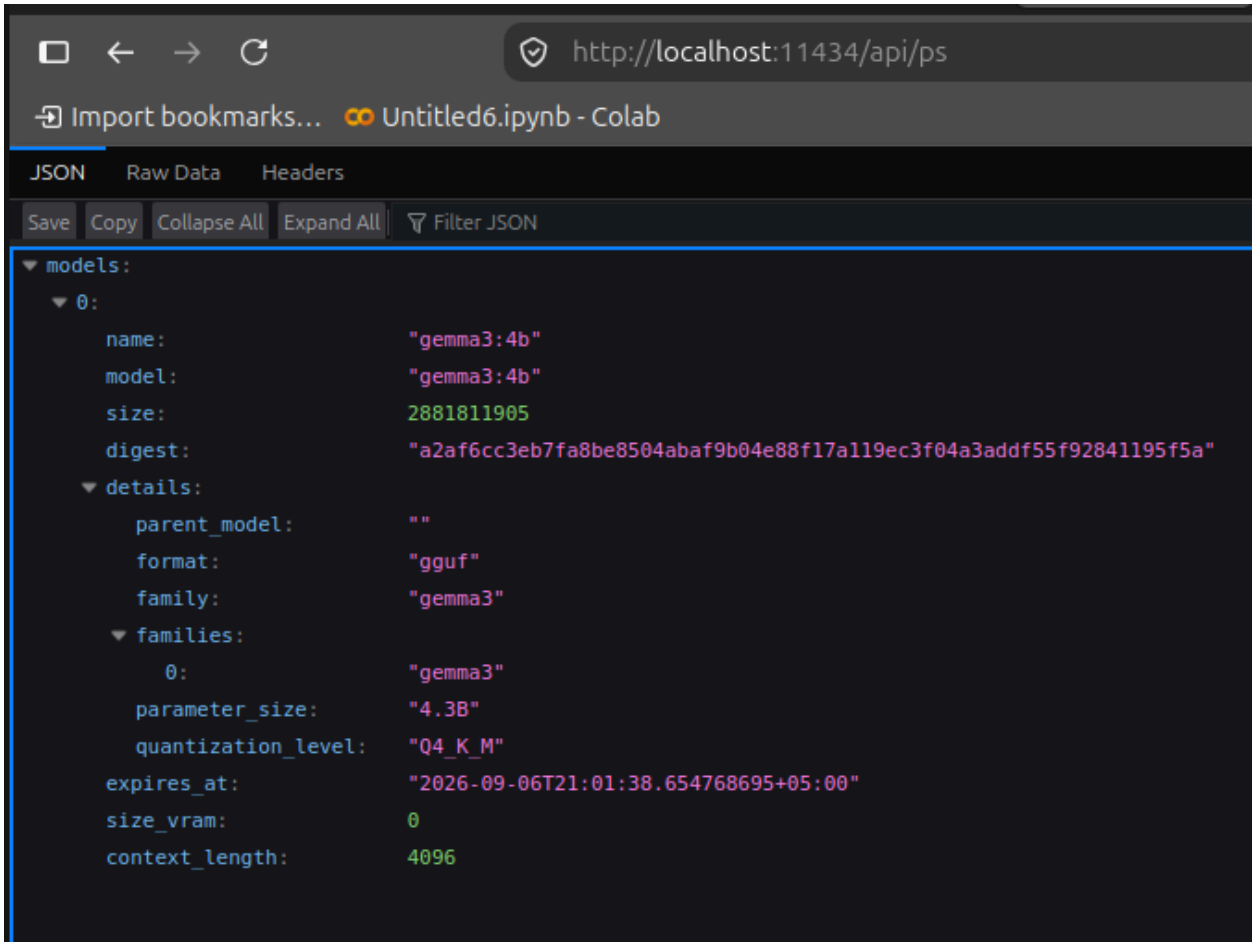

CONNECTING
Deploy
Agent Settings
Local Model
gemma3:4b
Memory (Past Runs)
No prior history in memory.
TrendPilot: Local Agentic Content Strategist
Powered by Local Ollama (Gemma 3:4B / Llama 3.2 3B) | AIRI Team PITB Task 2
Enter your Project or Topic:
YOLOv8 helmet safety detection project
Target Platform:
LinkedIn
Tone:
Professional + Exciting
Generate Complete Viral Plan
Agent completed all tasks successfully!
Output saved locally to: outputs/saved_results/YOLOv8_helmet_safety_dete_20260906_210142.md
Caption & Hook
Video Script
Hashtags
Review & Plan
Raw Markdown
Selected Hashtags
#AI #ComputerVision #YOLOv8 #HelmetDetection #ObjectDetection #SafetyTech #AutonomousVehicles #Robotics #DeepLearning #VisionAI #YOLOv8Helmet

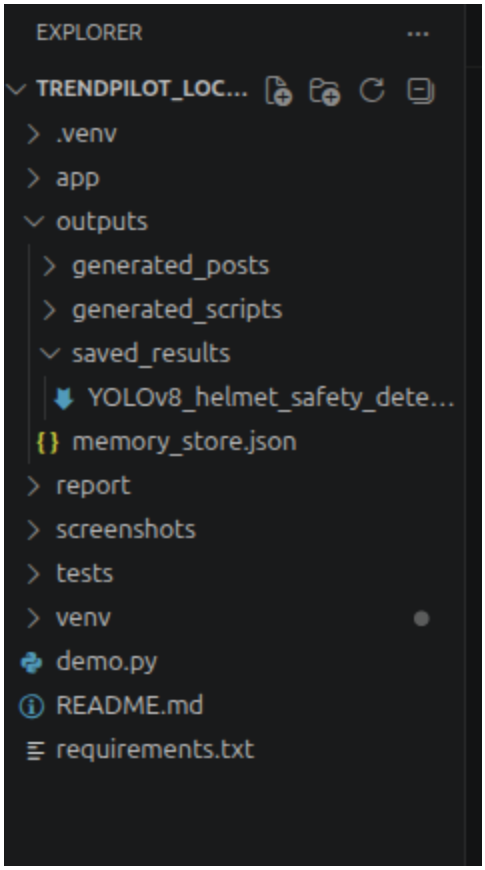

CONNECTING
Deploy
Agent Settings
Local Model
gemma3:4b
Memory (Past Runs)
No prior history in memory.
Caption & Hook
Video Script
Hashtags
Review & Plan
Raw Markdown
Execution Plan
Okay, here's a 4-step plan for a viral LinkedIn campaign around YOLOv8 helmet detection:
1. **Angle:** Position as *innovative safety tech* – highlighting real-time detection & impact.
2. **Hook/Caption:** Craft a compelling question/statistic + concise explanation of YOLOv8's value (e.g., "Is your safety tech *really* seeing it?").
3. **Media:** Develop a short video showcasing the detection in action, overlaid with clear data visualizations, aiming for 60-90 seconds.
4. **Optimization:** Research & utilize relevant hashtags (#AIsafety, #wearabletech, #YOLOv8) and rigorously QA all content for accuracy & brand voice.
Reviewer QA Feedback
Okay, here's a review of the draft LinkedIn content, followed by a scoring and actionable recommendations:
Overall Review:
This draft is a solid starting point for LinkedIn content about the aimi project. The three options offer variety, and the core information – the technology, the problem it solves, and the call to action – are all present. However, it leans a bit heavily on technical jargon and could be significantly strengthened to maximize engagement on LinkedIn, which tends to favor more accessible and visually driven content. The hooks, while present, need a sharper, more immediate punch.
Scoring (1-10):
Hook Strength: 6/10 – The hooks are functional but could be much more compelling. They're a bit dry and descriptive rather than intriguing.
Readability: 8/10 – The writing is clear and understandable, especially Option 3. However, the longer captions could benefit from tighter phrasing and more frequent use of bullet points.
Platform Fitness: 6/10 – It's appropriate for LinkedIn, but lacks the visual emphasis and conversational tone that performs best on the platform.
Recommendations for 10x Improvement (2 Bullet Points):
Hyper-Focus the Hooks with a Problem-Oriented Question: Instead of stating the solution ("We're building the future of safety…"), immediately grab attention by posing a relatable problem. For example, for Option 1: "Are you frustrated with inaccurate helmet detection? aimi is here to change that." Or, for Option 3: "What if helmet detection could actually save lives? 🚀" LinkedIn users respond extremely well to content that directly addresses their pain points.
Prioritize Visuals & Short-Form Content: LinkedIn is a visual platform. Immediately emphasize this. Suggest a shift toward creating short video demos (15-30 seconds) demonstrating aimi in action, or a series of visually striking graphics explaining the technology simply. Consider repurposing the longer captions into a series of shorter posts (think 1-2 sentences for initial engagement, followed by a link to the full content). A single, powerful image or video dramatically outperforms a long caption.
Further Considerations & Suggestions:
Quantify the Benefits More Clearly: While "25% improvement" is good, consider framing it in terms of lives saved or reduced injuries. Numbers resonate deeply with LinkedIn users.
Show, Don't Just Tell: The descriptions of "cutting-edge" and "revolutionary" are buzzwords. Provide concrete examples of where and how aimi is being used (even if it's simulated data or a pilot


CONNECTING
Deploy
Agent Settings
Local Model
gemma3:4b
Memory (Past Runs)
No prior history in memory.
Caption & Hook
Video Script
Hashtags
Review & Plan
Raw Markdown
TrendPilot Output: YOLOv8 helmet safety detection project
- **Platform:** LinkedIn
- **Tone:** Professional + Exciting
- **Generated At:** 2026-09-06 21:01:42

🎯 Viral Angle & Plan
Okay, here's a 4-step plan for a viral LinkedIn campaign around YOLOv8 helmet detection:
1. **Angle:** Position as *innovative safety tech* – highlighting real-time detection & impact.
2. **Hook/Caption:** Craft a compelling question/statistic + concise explanation of YOLOv8's value (e.g., "Is your safety tech *really* seeing it?").
3. **Media:** Develop a short video showcasing the detection in action, overlaid with clear data visualizations, aiming for 60-90 seconds.
4. **Optimization:** Research & utilize relevant hashtags (#AIsafety, #wearabletech, #YOLOv8) and rigorously QA all content for accuracy & brand voice.

🎣 Hook
> Okay, here are a few LinkedIn content angles for the YOLOv8 helmet safety detection project – "aimi," aiming for a professional yet exciting tone. I've created th
---
📝 Caption
Okay, here are a few LinkedIn content angles for the YOLOv8 helmet safety detection project – "aimi," aiming for a professional yet exciting tone. I've created thre
---
Option 1: Focus on Impact & Problem Solving
1. Viral Hook:
"Is your city's traffic safety truly *seeing* everyone wearing a helmet? We're building the future of safety with aimi – a revolutionary AI that's dramatically impr
2. Body Caption:
"Road safety is a critical global challenge, and the number of preventable head injuries remains alarming. Traditional helmet detection methods are often inaccurate
Developed using the cutting-edge YOLOv8 object detection model, aimi provides unparalleled accuracy in identifying helmets – even in challenging conditions like low


http://localhost:11434/api/ps
Import bookmarks...
Untitled6.ipynb - Colab
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
models:
0:
name: "gemma3:4b"
model: "gemma3:4b"
size: 2881811905
digest: "a2af6cc3eb7fa8be8504abaf9b04e88f17a119ec3f04a3addf55f92841195f5a"
details:
parent_model: ""
format: "gguf"
family: "gemma3"
families:
0: "gemma3"
parameter_size: "4.3B"
quantization_level: "Q4_K_M"
expires_at: "2026-09-06T21:01:38.654768695+05:00"
size_vram: 0
context_length: 4096



Testing Suite & Validation

To ensure the agentic workflow was robust across different domains and social platforms, the system was subjected to 10 distinct test cases. The tests spanned technical AI topics, cybersecurity workflows, and general student-life narratives to validate the prompt engineering and tool reliability.

Agent Test Case Matrix

Test No.	Input Topic	Target Platform	Requested Tone	Expected Output	Status

1	YOLOv8 helmet safety detection project	LinkedIn	Professional + Exciting	Ideas, Caption, Script, Tags, QA, Markdown File	Passed
2	Using FinalRecon for domain footprinting in Kali Linux	X / Twitter	Technical & Educational	Short tip, high-value angle, strict character awareness	Passed
3	Debugging PyTorch CUDA out of memory errors	Instagram Reels	Humorous / Meme Style	Relatable script with visual/audio meme cues	Passed
4	Comparing RT-DETR vs YOLO object detection	LinkedIn	Professional + Exciting	Technical breakdown formatted cleanly with hashtags	Passed
5	6-month AI Internship Lessons at PITB	LinkedIn	Storytelling / Vulnerable	Personal narrative hook, reflective body caption	Passed
6	Deploying MLflow tracking server using Docker	YouTube Shorts	Technical & Educational	Fast-paced script explaining ports and containers	Passed

7	Model worked in Dev, Failed in Production	TikTok	Humorous / Meme Style	Funny relatable 30s video script with audio notes	Passed
8	Configuring UFW firewall rules on Ubuntu	X / Twitter	Technical & Educational	Concise thread intro, practical commands, tags	Passed
9	Data Labeling struggles (annotating 10k images)	Instagram Reels	Storytelling / Vulnerable	Emotive script focusing on the grind of dataset prep	Passed
10	AI proctoring system using OpenCV	LinkedIn	Professional + Exciting	Strong project showcase angle with clear CTA	Passed

Error Analysis & System Optimization

Building an AI agent with local hardware requires significant optimization. Smaller LLMs are highly efficient but occasionally struggle with strict formatting constraints. Reviewing the initial failed or weak outputs during testing was critical to improving the system architecture.

Below is an analysis of system failures encountered during development and the engineering fixes applied to solve them.

System Failure Analysis & Applied Fixes

Test Case / Failure Mode	**Problem Found**	**Root Cause Analysis**	**Engineering Fix Applied**

File Saving Crash (Initial YOLOv8 run)	The application threw an OS-level FileNotFoundError and crashed when attempting to save the markdown file.	The user input string contained special characters (e.g., colons, spaces) which are invalid for file naming in Linux/Windows file systems.	Implemented Regex sanitization (re.sub) in the tool_file_saver to strip special characters and limit filenames to 25 chars.
Timeout Exception (All large outputs)	Streamlit threw a Read timed out error after exactly 1 minute of processing, yielding no output.	The local 4B parameter model required more than 60 seconds to generate long text chunks on local hardware. Python's default request timeout killed the socket.	Modified the requests.post() call in app/tools.py to include timeout=300, allowing the model 5 full minutes to compute.

Hashtag Clutter (Tests 2 & 4)	The Hashtag tool returned conversational text: *"Here are your hashtags: #AI #ML"*.	Local models exhibit "chatty" bias and sometimes ignore instructions demanding *only* data output.	Bypassed prompt engineering by implementing programmatic Regex parsing (re.findall) to strictly extract strings beginning with #.
X/Twitter Post Length (Test 8)	The generated Twitter post was over 600 words long, ignoring platform limits.	Small models default to writing blog-style paragraphs unless heavily constrained by the system prompt.	Updated the CAPTION_WRITER_PROMPT to enforce strict formatting and character awareness specifically when the platform equals "X".
Meme Script Pacing (Test 3 & 7)	The generated short-form video scripts were too text-heavy and read like essays rather than fast-paced videos.	The LLM lacked a structured temporal framework for scripting 30-second shorts.	Added rigid pacing brackets to the system prompt ([0:00-0:05] Hook, [0:05-0:20] Problem) to force temporal brevity.

Key Learnings & Future Improvements

Key Learnings

Building TrendPilot provided hands-on experience bridging the gap between theoretical LLM interaction and practical software engineering. Key takeaways include:

1. **The Distinction Between Chatbots and Agents:** I learned that Agentic AI is fundamentally about task decomposition. Instead of asking a model to "write a post," an

agent requires a systematic orchestration of planning, deterministic tool dispatch, and state tracking.
2. **Local AI Deployment Logistics:** Successfully running gemma3:4b via Ollama highlighted the viability of local, privacy-first AI. However, it also taught me to anticipate and handle hardware constraints, specifically managing API timeout thresholds for longer inference loads.
3. **Prompt Engineering vs. Programmatic Guardrails:** I learned that while prompt engineering is powerful, it is not flawless—especially with 3B/4B parameter models. Using Python regular expressions (like in the Hashtag tool) to enforce deterministic constraints is far more reliable than begging the LLM to output clean data.

Future Improvements

Given additional development time, the following architectural upgrades would be implemented:

1. **Multi-Agent LangGraph Implementation:** Transition the linear Python script to a cyclic graph architecture where a "Planner Agent" and "Writer Agent" can argue and iterate multiple times before outputting the final draft.
2. **Web Search Tool Integration:** Integrate the DuckDuckGo API as an external tool, allowing the Trend Idea Generator to scrape current, real-world trending topics before drafting angles.
3. **Database Memory Upgrade:** Upgrade the simplistic JSON memory system to a robust local SQLite database to allow for querying past campaigns by platform or tone.